

相机与图像 (Camera and Image)

王永才



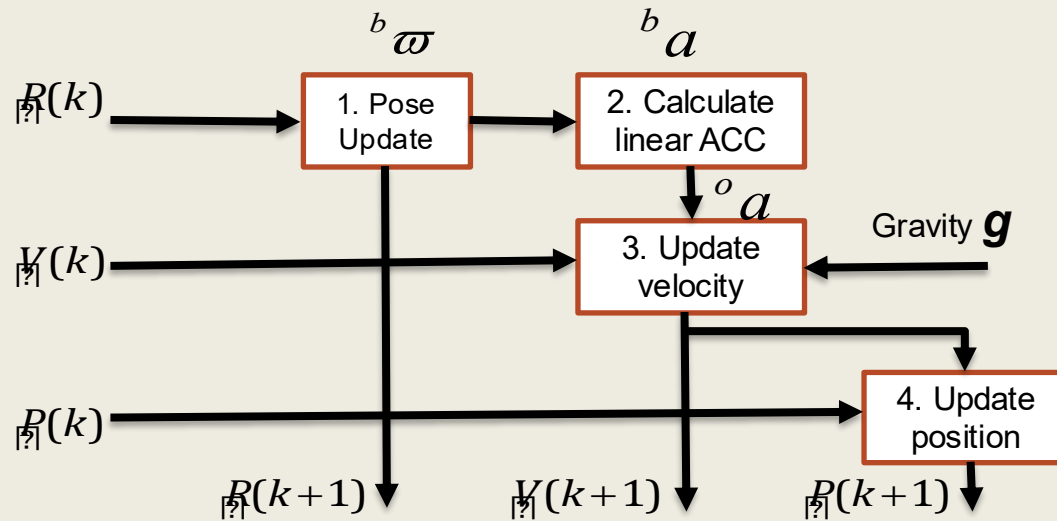
目录

相机和图像

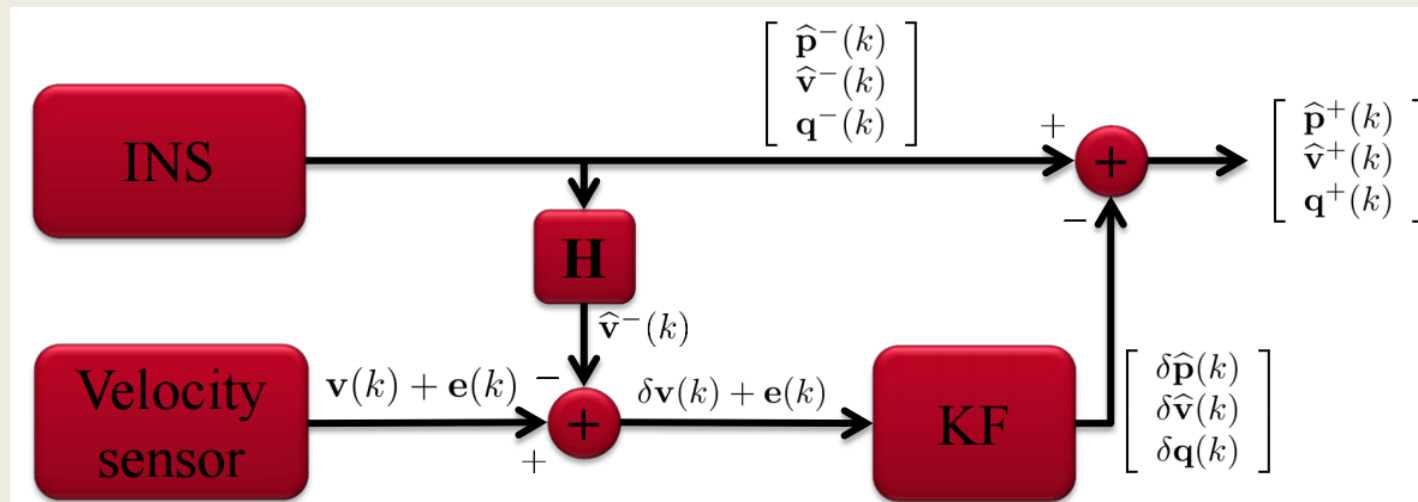
特征点提取与匹配

2D-2D对极几何

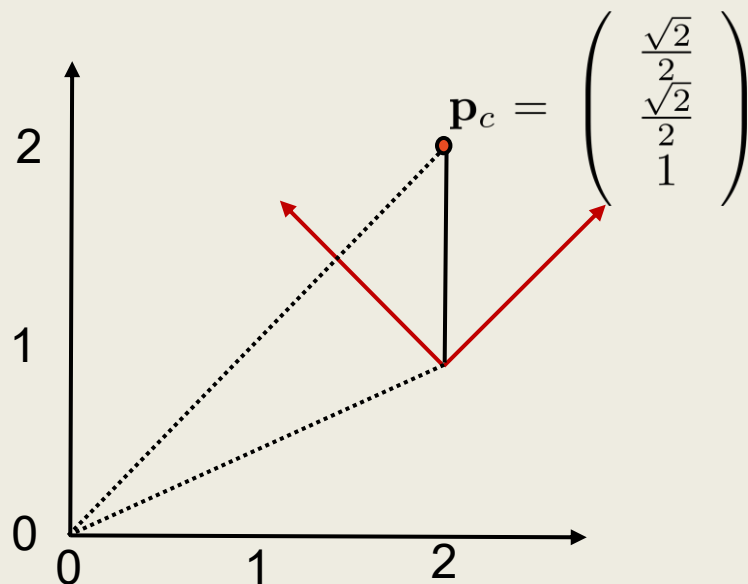
上节课内容：惯性导航



追踪的移动目标
在世界坐标系下
的位置和姿态



INS追踪的究竟是什么? T_{wc}



相机在世界坐标系下的
位置和姿态

$$p = [2, 1]^T, \theta = \frac{\pi}{4}$$

所对应的变换矩阵是：世界坐标系变换到相机坐标系的变换矩阵

$$T_{wc} = \begin{bmatrix} \mathbf{R}_{wc} & t \\ 0^T & 1 \end{bmatrix} = \begin{pmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} & 2 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

T_{wc} 可将相机坐标系下点的坐标，变换为其世界坐标系坐标

$$\mathbf{p}_w = T_{wc} \mathbf{p}_c = \begin{pmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} & 2 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix}$$

追踪的变换矩阵与坐标系变换

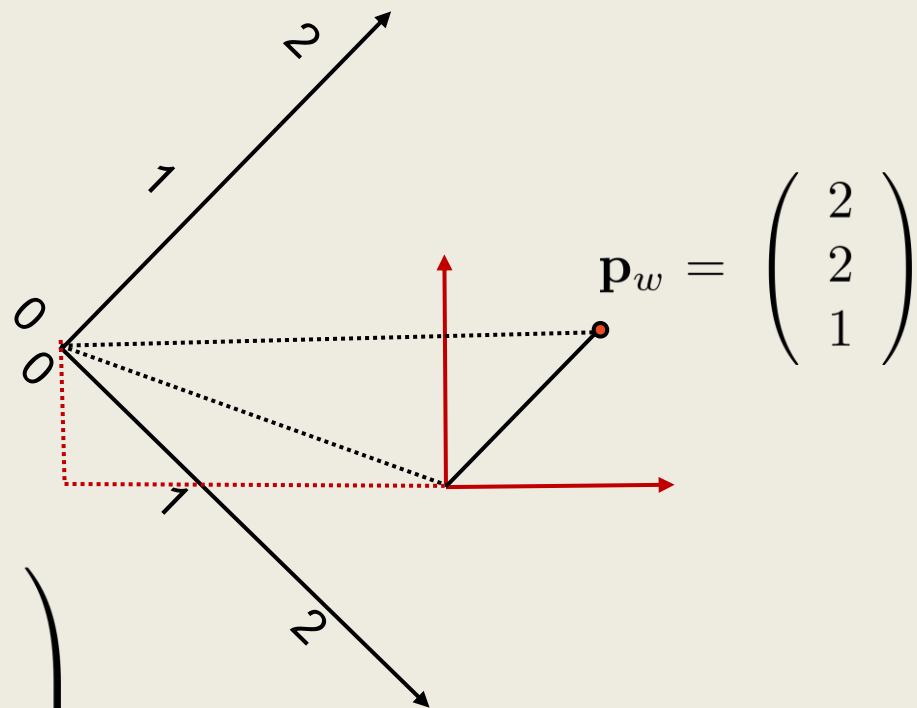
$$\mathbf{T}_{cw} = \mathbf{T}_{wc}^{-1}$$

相机坐标系变换到世界坐标系所需要的经历的变换

$$\mathbf{T}_{cw} = \mathbf{T}_{wc}^{-1}$$

$$= \begin{pmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & -\sqrt{\frac{9}{2}} \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{p}_c = \mathbf{T}_{cw} \mathbf{p}_w = \begin{pmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & -\sqrt{\frac{9}{2}} \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \\ 1 \end{pmatrix}$$

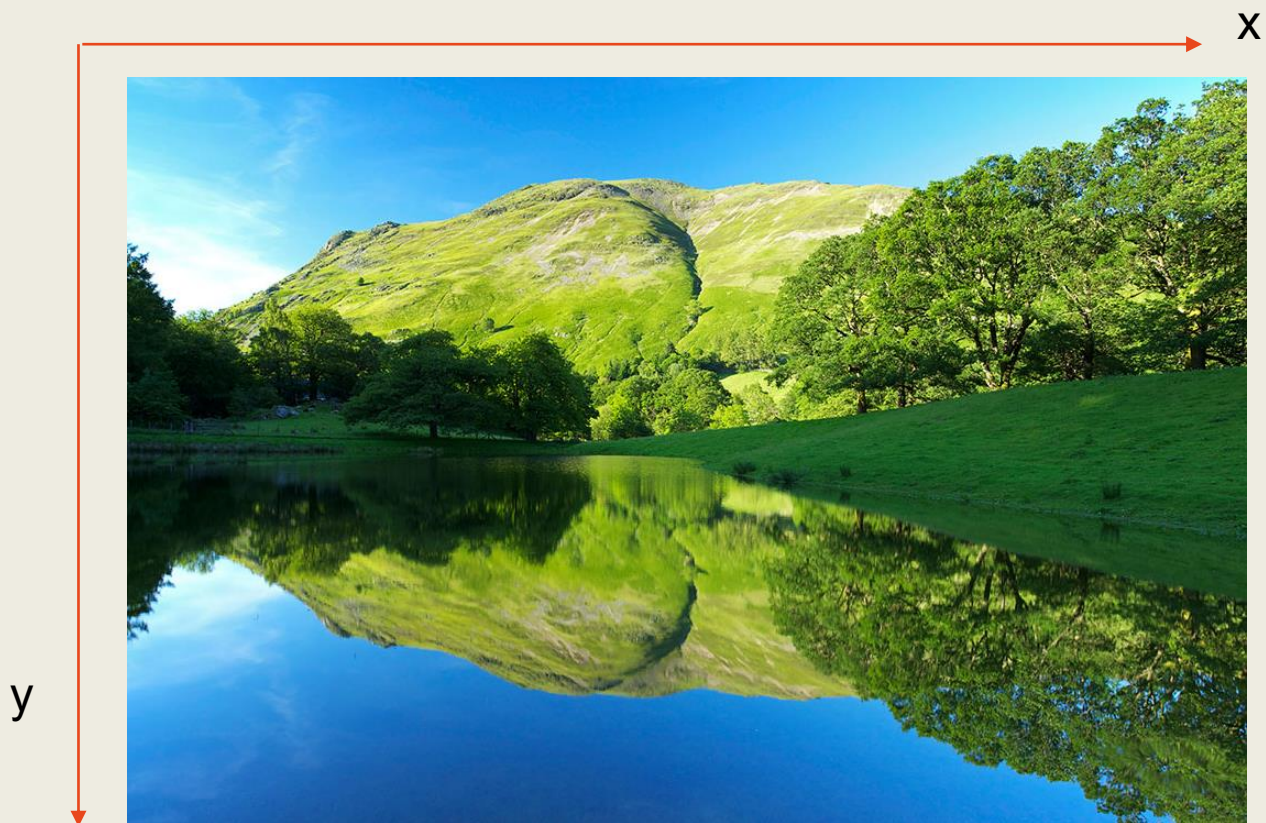


本讲目标

- 理解针孔相机的模型、内参与径向畸变参数。
- 理解一个空间点是如何投影到相机成像平面的。
- 掌握OpenCV的图像存储与表达方式。
- 学会基本的摄像头标定方法。

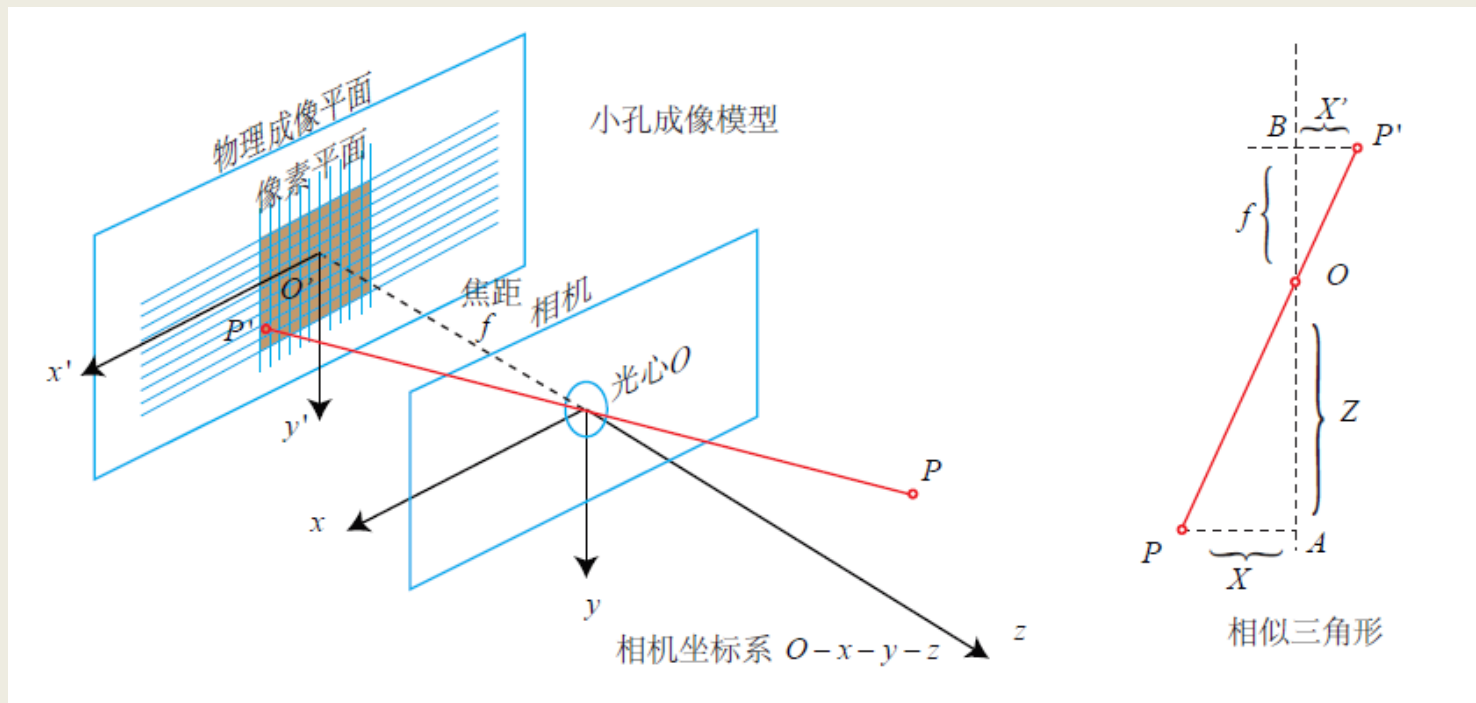
相机模型

- 我们在现实生活中看到过大量的照片
 - 照片记录了真实世界在成像平面上的投影
 - 这个过程丢弃了“距离”维度上的信息
 - 普通相机可以用针孔模型很好地近似



相机模型

- 小孔成像模型



原始形式

$$\frac{Z}{f} = -\frac{X}{X'} = -\frac{Y}{Y'}.$$

翻转到前面

$$\frac{Z}{f} = \frac{X}{X'} = \frac{Y}{Y'}.$$

整理之：

$$\begin{aligned} X' &= f \frac{X}{Z} \\ Y' &= f \frac{Y}{Z} \end{aligned}.$$

相机模型

- 小孔成像模型
- 成像平面到像素坐标

α, β 由传感器的尺寸决定，每mm传感器尺寸对应的像素值

c_x, c_y 由图像的分辨率决定， $W/2, H/2$

f 焦距，相平面到光心的距离

$$\begin{cases} u = \alpha X' + c_x \\ v = \beta Y' + c_y \end{cases}$$

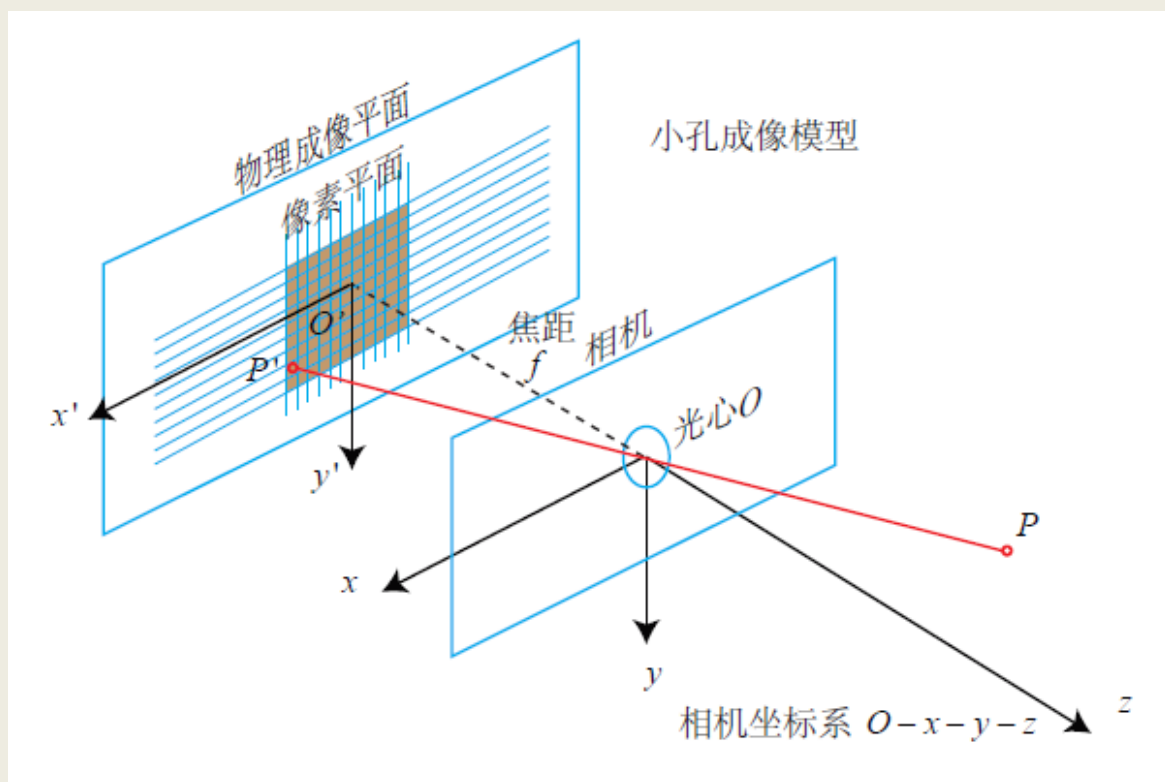
代入

$$\begin{cases} X' = f \frac{X}{Z} \\ Y' = f \frac{Y}{Z} \end{cases}$$

得

$$\begin{cases} u = f_x \frac{X}{Z} + c_x \\ v = f_y \frac{Y}{Z} + c_y \end{cases}$$

$$f_x = \alpha f \quad f_y = \beta f$$



相机模型

展开形式

$$\begin{cases} u = f_x \frac{X}{Z} + c_x \\ v = f_y \frac{Y}{Z} + c_y \end{cases}.$$

矩阵形式

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{Z} \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \triangleq \frac{1}{Z} K P.$$

传统习惯

$$Z \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \triangleq K P.$$

左侧是齐
次坐标

中间矩阵称
为内参数

右侧是非
齐次坐标

内参通常在相机生产之后就已固定

相机的内参矩阵

- KITTI数据集中，calib.txt：摄像机的校准数据。
- P_0 ， P_1 是校正后的3x4投影矩阵。这里 P_0 表示左摄像机， P_1 表示右摄像机。

$$P_0 = \begin{bmatrix} f_x & 0 & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$P_0 = \text{array} \left(\begin{bmatrix} 707, & 0, & 602, & 0 \\ 0, & 707, & 183, & 0 \\ 0, & 0, & 1, & 0 \end{bmatrix} \right)$$

P0: 7.188e+02 0.00000e+00 6.07192e+02 0.00000e+00
 0.00000e+00 7.1885e+02 1.8521e+02 0.0000e+00
 0.0000e+00 0.0000e+00 1.0000e+00 0.0000e+00

P1: 7.1885e+02 0.0000e+00 6.0719280e+02 -3.8610e+02
 0.0000e+00 7.1885e+02 1.8521e+02 0.0000e+00
 0.0000e+00 0.000e+00 1.0000e+00 0.000e+00

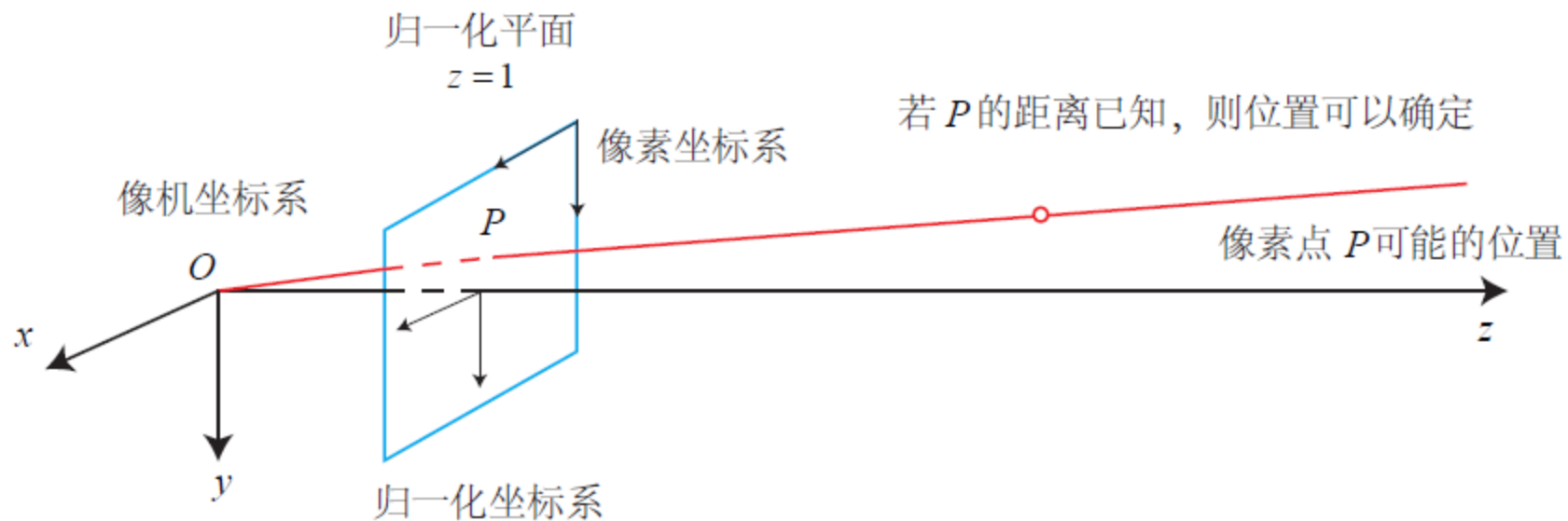
Tr: 4.276802385584e-04 -9.999672484946e-01 -8.084491683471e-03 -1.198459927713e-02 -7.210626507497e-03
 8.081198471645e-03 -9.999413164504e-01 -5.403984729748e-02 9.999738645903e-01 4.859485810390e-04 -
 7.206933692422e-03 -2.921968648686e-01

$$\mathbf{P}_{rect}^{(i)} = \begin{pmatrix} f_u^{(i)} & 0 & c_u^{(i)} & -f_u^{(i)} b_x^{(i)} \\ 0 & f_v^{(i)} & c_v^{(i)} & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

相机模型

$$Z \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \triangleq KP.$$

改变Z时，投影点仍是同一个



相机模型

- 除内参外，相机坐标系与世界坐标系还相差一个变换：

$$Z\mathbf{P}_{uv} = Z \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{K}(\mathbf{R}_{cw}\mathbf{P}_w + \mathbf{t}_{cw}) = \mathbf{K}\mathbf{T}_{cw}\mathbf{P}_w$$

先把 $\mathbf{P}_w$ 从世界坐标变到
相机坐标系下

- 这里 $\mathbf{R}_{cw}$, $\mathbf{t}_{cw}$ 或 $\mathbf{T}_{cw}$ 称为外参
 - 注意：相机模型中是 $\mathbf{T}_{cw}$ ，惯性导航追踪的结果是 $\mathbf{T}_{wc}$
 - 注意：右侧式子隐含了一次非齐次到齐次的变换（见书）
- 外参是SLAM估计的目标

相机模型

世界——相机——归一化平面——像素

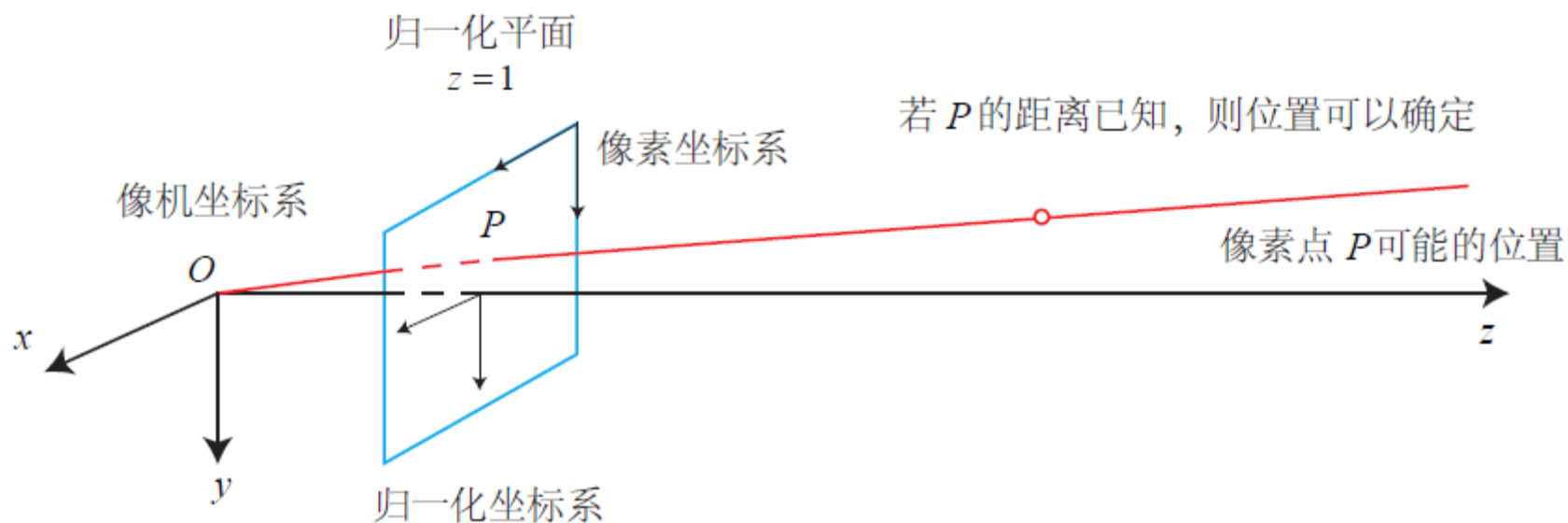
像素坐标

归一化平面坐标

相机坐标

世界坐标

$$Z\mathbf{P}_{uv} = Z \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{K}(\mathbf{R}_{cw}\mathbf{P}_w + \mathbf{t}_{cw}) = \mathbf{K}\mathbf{T}_{cw}\mathbf{P}_w$$



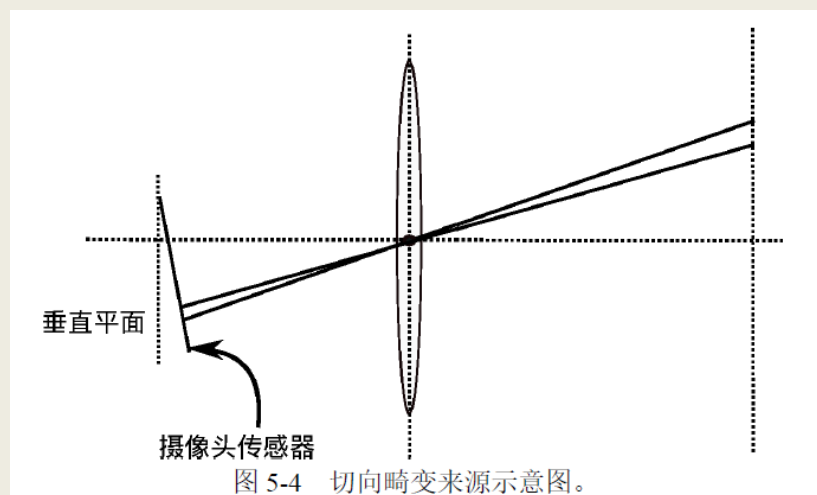
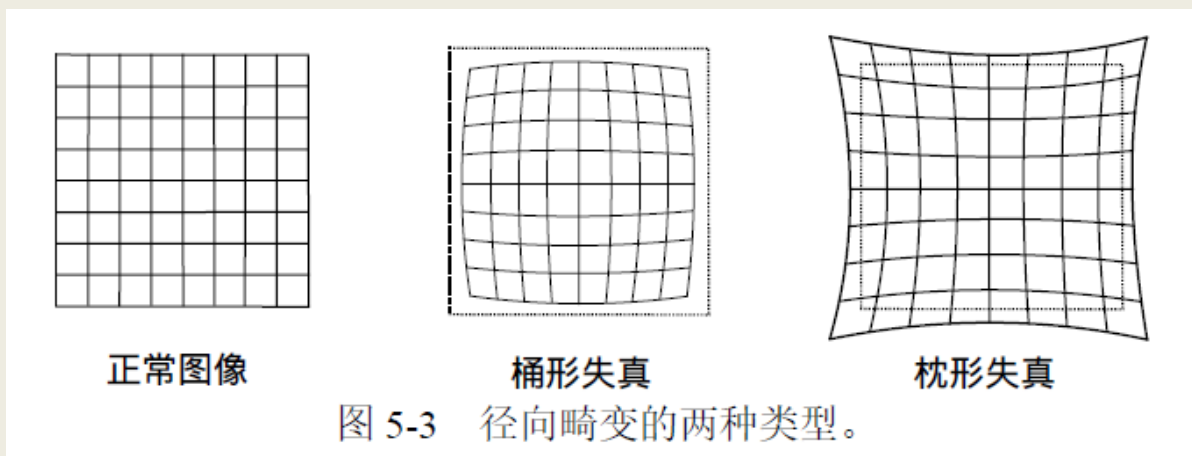
相机模型

- 畸变
 - 针孔前的镜头会引入畸变



相机模型

- 主要的畸变类型：径向畸变和切向畸变



相机模型

- 数学模型
 - 畸变可以用归一化坐标的变换来描述

$$\begin{aligned}x_{corrected} &= x(1 + k_1r^2 + k_2r^4 + k_3r^6) \\ y_{corrected} &= y(1 + k_1r^2 + k_2r^4 + k_3r^6)\end{aligned}$$

r 是距离中心的距离

径向畸变：多项式描述

$$\begin{aligned}x_{corrected} &= x + 2p_1xy + p_2(r^2 + 2x^2) \\ y_{corrected} &= y + p_1(r^2 + 2y^2) + 2p_2xy\end{aligned}$$

切向畸变：多项式描述

p_1, p_2 是纠正切向畸变的参数

放在一起：

实际当中可灵活保留各项系数

$$\begin{cases}x_{corrected} = x(1 + k_1r^2 + k_2r^4 + k_3r^6) + 2p_1xy + p_2(r^2 + 2x^2) \\ y_{corrected} = y(1 + k_1r^2 + k_2r^4 + k_3r^6) + p_1(r^2 + 2y^2) + 2p_2xy\end{cases}$$

相机模型

小结

1. 首先，世界坐标系下有一个固定的点 P ，世界坐标为 P_w ；
2. 由于相机在运动，它的运动由 R, t 或变换矩阵 $T \in SE(3)$ 描述。 P 的相机坐标为：
 $\tilde{P}_c = RP_w + t$ 。
3. 这时的 $\tilde{P}_c$ 仍有 X, Y, Z 三个量，把它们投影到归一化平面 $Z = 1$ 上，得到 P 的归一化相机坐标： $P_c = [X/Z, Y/Z, 1]^T$ ^①。
4. 最后， P 的归一化坐标经过内参后，对应到它的像素坐标： $P_{uv} = KP_c$ 。

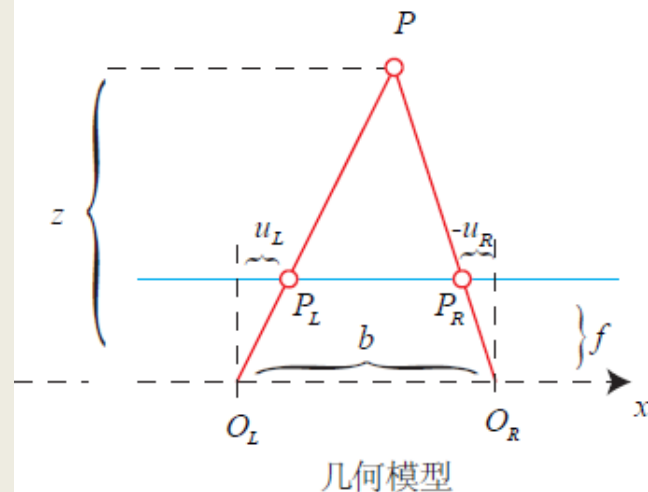
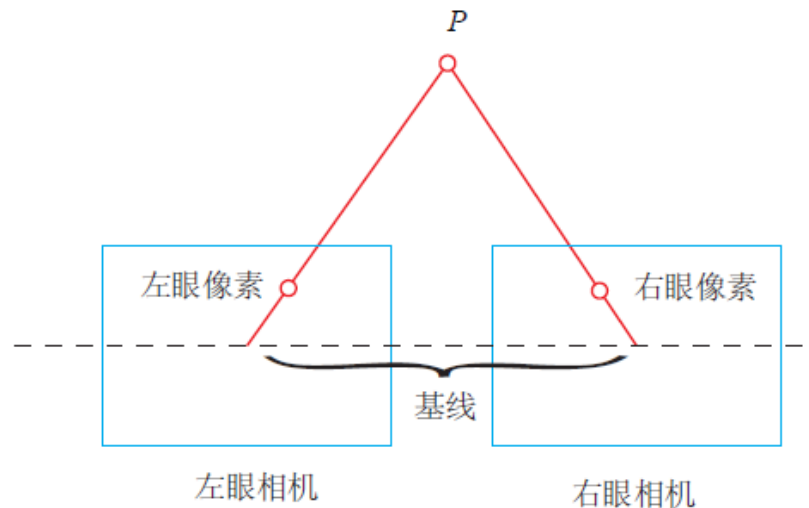
双目相机模型

- 左右相机中心距离称为基线
- 左右像素的几何关系：

$$\frac{z - f}{z} = \frac{b - u_L + u_R}{b}.$$

- 整理得

$$z = \frac{fb}{d}, \quad d = u_L - u_R.$$



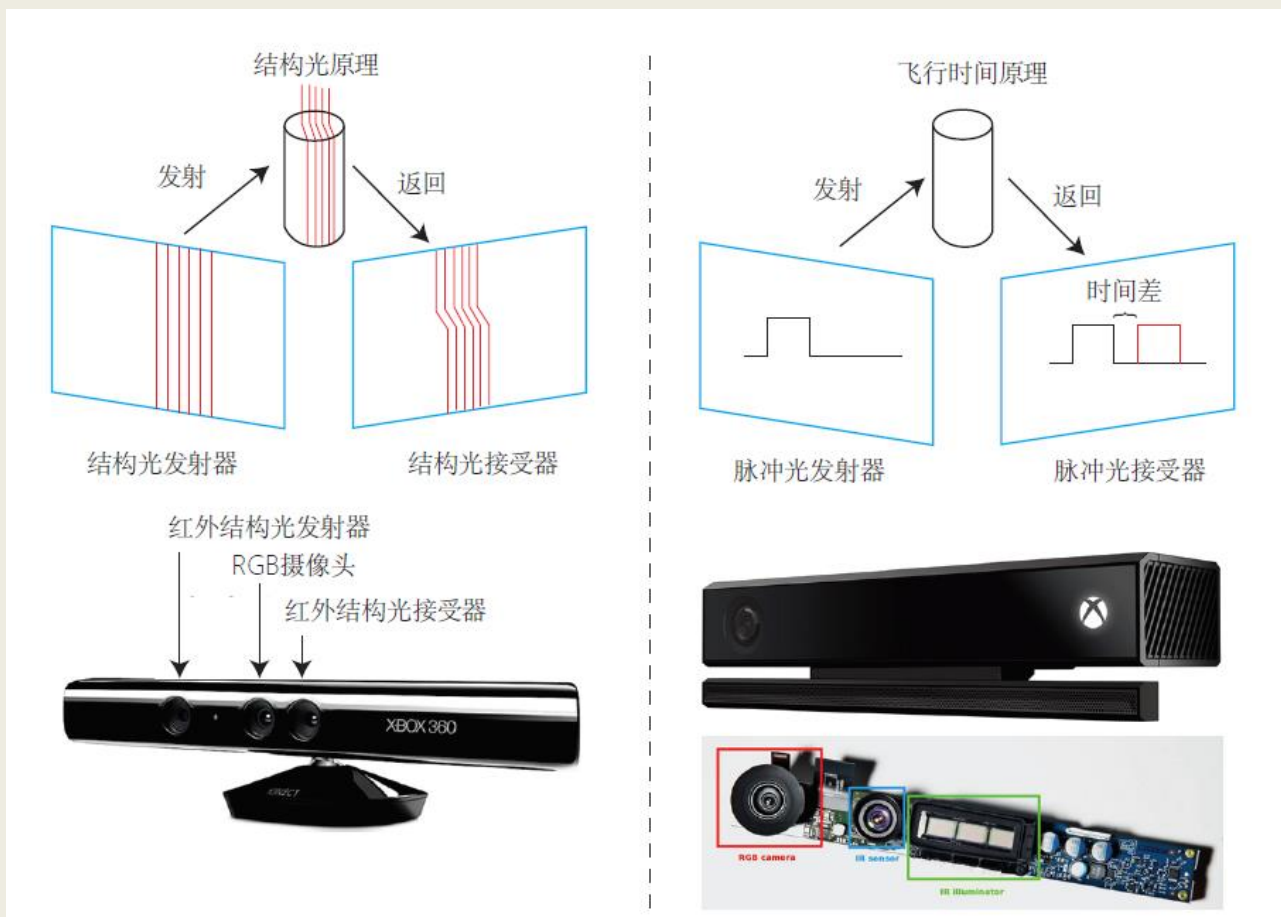
d称为视差（disparity），描述同一个点在左右目上成像的距离

d最小为1个像素，因此双目能测量的z有最大值：fb

双目相机能直接获得目标深度，距离公式简单，但d不容易计算

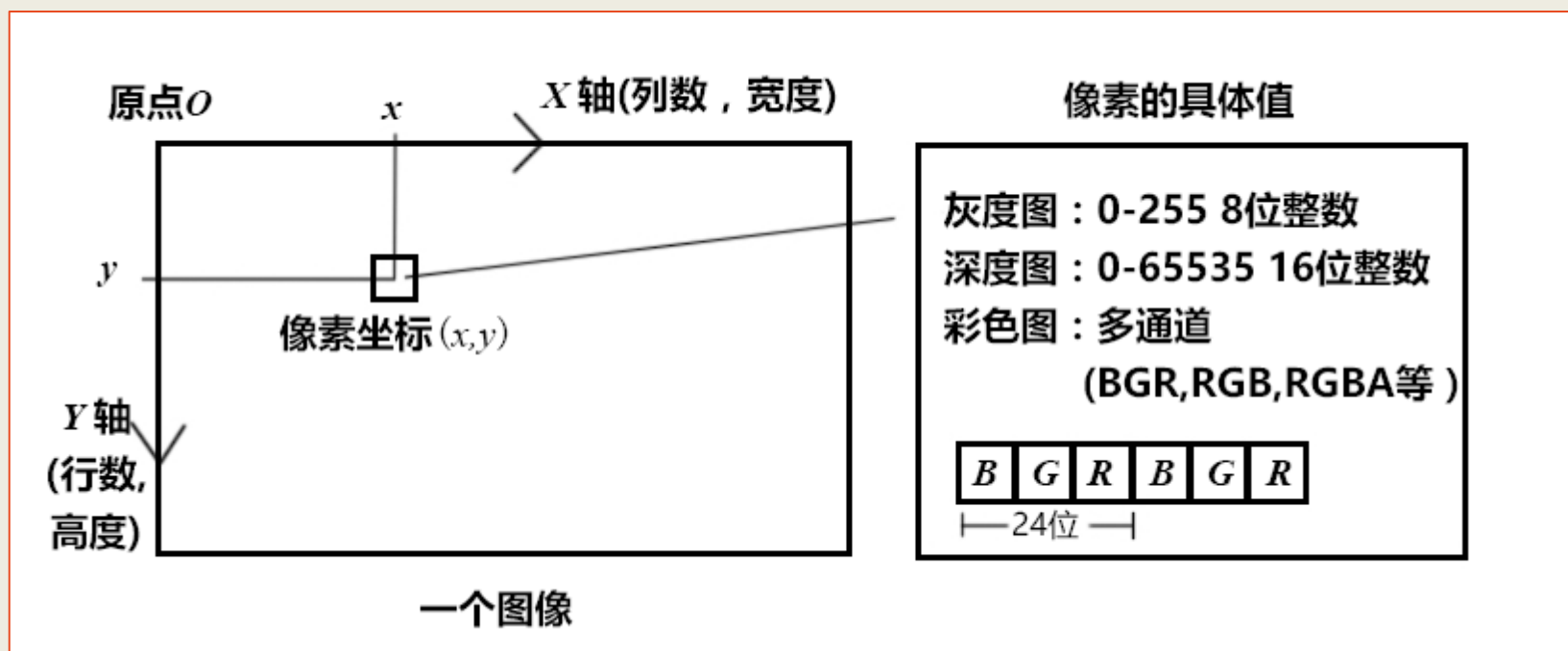
RGB-D 相机模型

- RGB-D相机：物理手段测量深度
 - ToF或结构光两种主要原理
 - 通常能得到与RGB图对应的深度图



图像

- 相机成像后，生成了图像
- `unsigned char image[480][640]`,
- 图像在计算机中以矩阵形式存储（二维数组）
 - 需要对感光度量化成数值，例如0~255之间的整数（彩色图像还有通道）
 - `unsigned char pixel = image[y][x]`



应用示例：opencv基本图像操作

// 读取argv[1]指定的图像

```
cv::Mat image;
```

```
image = cv::imread ( argv[1] ); //cv::imread函数读取指定路径下的图像
```

// 显示Image图像的信息

```
cout<<"图像宽为"<<image.cols<<",高为"<<image.rows<<",通道数为  
"<<image.channels())<<endl;
```

```
cv::imshow ( "image", image );    // 用cv::imshow显示图像
```

//用image_clone拷贝图像，用setTo修改图像像素值

```
cv::Mat image_clone = image.clone();
```

```
image_clone ( cv::Rect ( 0,0,100,100 ) ).setTo ( 255 );
```

```
cv::imshow ( "image", image );
```

应用示例：拼接点云

$$\begin{cases} u = f_x \frac{X}{Z} + c_x \\ v = f_y \frac{Y}{Z} + c_y \end{cases}$$

```
Eigen::Vector3d point;  
point[2] = double(d)/depthScale;  
point[0] = (u-cx)*point[2]/fx;  
point[1] = (v-cy)*point[2]/fy;
```

获得的是相机坐标系的点的坐标。

将相机坐标系点坐标，变换到世界坐标系：

```
Eigen::Vector3d pointWorld = Twc*point;
```

//T_{wc}是世界坐标系变换到相机坐标系的变换矩阵

//这个例子中给的是T_{wc}: $x, y, z, q_x, q_y, q_z, q_w$

```
Eigen::Quaterniond q( data[6], data[3], data[4], data[5] );
```

```
Eigen::Isometry3d T(q);
```

```
T.pretranslate( Eigen::Vector3d( data[0], data[1], data[2] ));
```

$$z = d$$

$$x = \frac{u - c_x}{f_x} \cdot z$$

$$y = \frac{v - c_y}{f_y} \cdot z$$

构造点云图

```
for ( int v=0; v<color.rows; v++ )  
    for ( int u=0; u<color.cols; u++ )  
        p.x = pointWorld[0]; //世界坐标系位置  
        p.y = pointWorld[1];  
        p.z = pointWorld[2];  
        p.b = color.data[ v*color.step+u*color.channels() ];  
        p.g = color.data[ v*color.step+u*color.channels()+1 ];  
        p.r = color.data[ v*color.step+u*color.channels()+2 ];  
        pointCloud->points.push_back( p );
```

//color.step是返回该Mat对象一行所占的数据字节数，此例中为
480*3=1920。

//color.channels() = 3

构造点云图

修改为如下代码更为直观：

```
for ( int v=0; v<color.rows; v++ )  
    for ( int u=0; u<color.cols; u++ )  
        p.x = pointWorld[0]; //世界坐标系位置  
        p.y = pointWorld[1];  
        p.z = pointWorld[2];  
        p.b = color.at<cv::Vec3b>(v, u)[0]; //B  
        p.g = color.at<cv::Vec3b>(v, u)[1]; //G  
        p.r = color.at<cv::Vec3b>(v, u)[2]; //R  
        pointCloud->points.push_back( p );
```



特征点提取与匹配 (Feature Extraction and Matching)

王永才

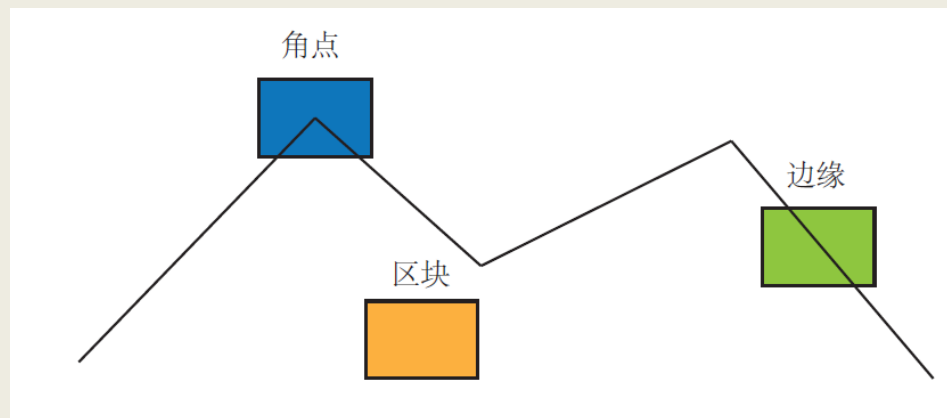
特征点提取与匹配

通过对齐相机在不同位置拍摄的照片，追踪相机的空间位置和姿态变化。

- 本讲目标
 - ORB特征点的概念
 - 掌握在单幅图像中提取出ORB特征点，及多幅图像中匹配特征点的方法。
 - 理解对极几何的原理，利用对极几何的约束，恢复出图像之间的摄像机的三维运动。

特征点法

- 经典SLAM模型中以位姿——路标（Landmark）来描述SLAM过程
- 路标是三维空间中固定不变的点，能够在特定位姿下观测到
 - 数量充足，以实现良好的定位
 - 较好的区分性，以实现数据关联
- 在视觉SLAM中，可利用图像特征点作为SLAM中的路标
- 特征点：图像当中具有代表性的部分
 - 可重复性
 - 可区别性
 - 高效
 - 本地



特征描述应该在光照、视角发生少量变化时
仍能保持一致

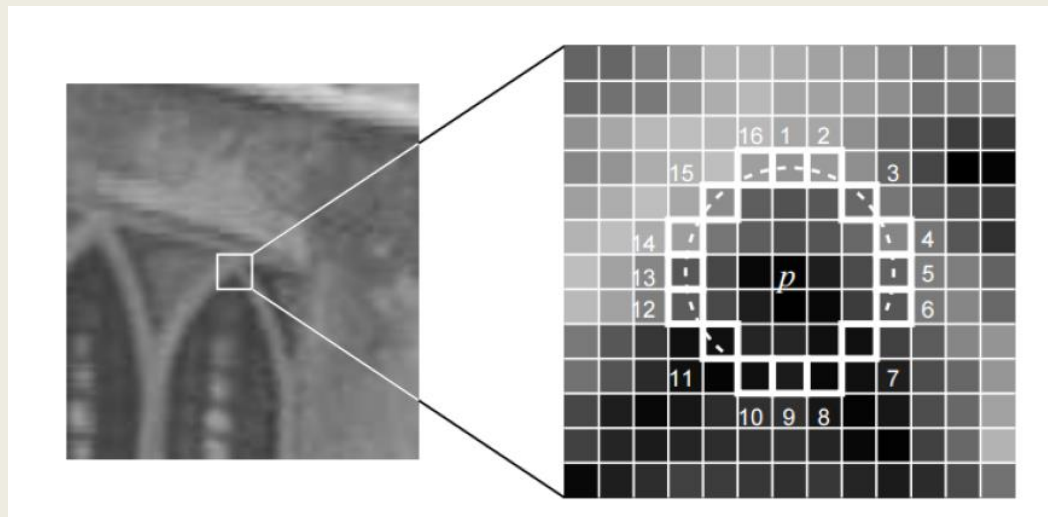
特征点法

特征点有两部分信息组成：

- 关键点（Key point）
 - 位置、大小、方向、评分等
 - SIFT（Scale-Invariant Feature Transform），尺度不变特征，计算很慢
 - SURF（Speeded Up Robust Feature），加速稳健特征
 - FAST（快速特征），计算特别快
- 描述子（Descriptor）
 - 描述子通常是一个向量，描述该关键点周围像素的信息；
 - 描述子是按照“外观相似的特征点应该有相似的描述子设计的”；
 - 在向量空间相近的两个描述子，认为是可能相同的特征点。
 - 常见描述子BRIEF(Binary Robust Indendent Elementary Feature)

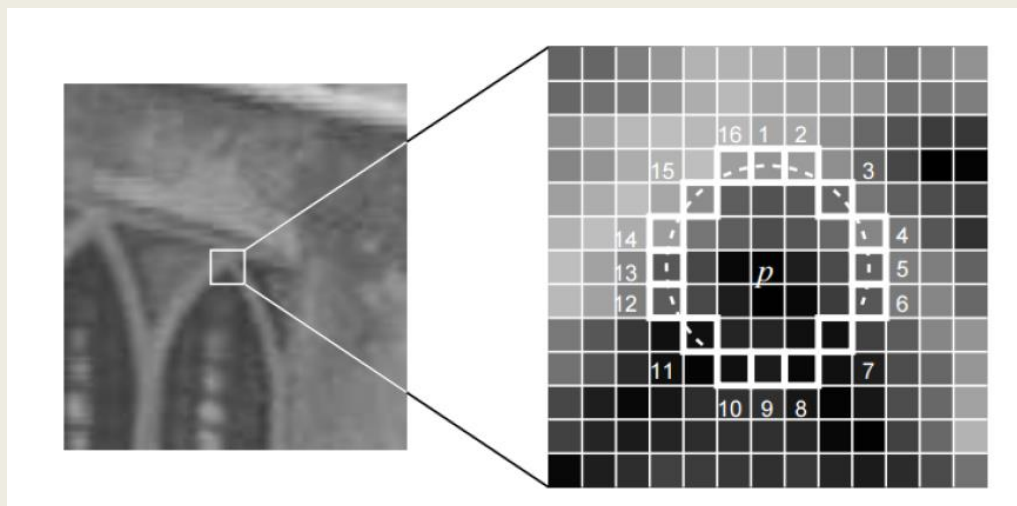
ORB特征

- ORB (Oriented FAST and Rotated BRIEF)
 - 关键点: Oriented FAST
 - 描述: BRIEF
- FAST
 - 连续N个点的灰度有明显差异
- Oriented FAST
 - 在FAST基础上计算旋转



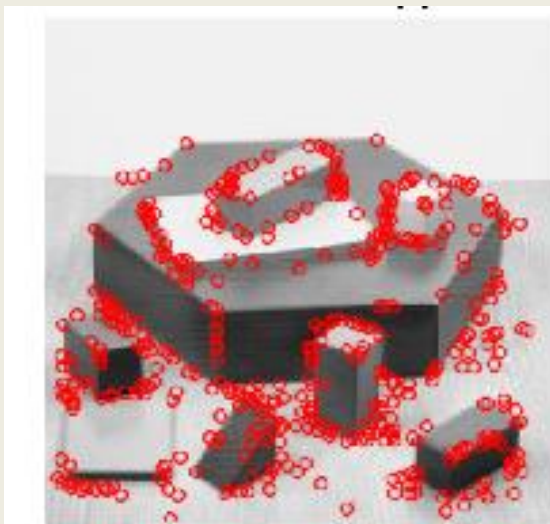
FAST关键点

- FAST是一种角点，主要检测灰度变化明显的地方，以速度快著称。
- 如果一个像素与邻域亮度差别特别大，它很可能是角点。
- 检测过程如下：
 - 对每个像素 p ，设亮度为 I_p
 - 以 p 为圆心，选择半径为3的圆上的16个邻居像素，如果有连续 N 个点的亮度大于 I_p+T 或小于 I_p-T ，则 p 是特征点。
 - N 一般取9,11,12
 - T 一般取20%

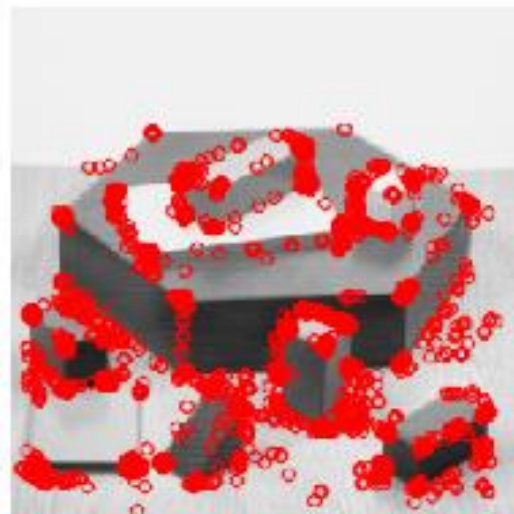


原始的FAST角点经常出现扎堆现象，还需要用非极大值抑制，在一定范围内仅保留响应极大值的角点。

FAST关键点



非极大值抑制之后



非极大值抑制之前

FAST特征点的问题：

- 数量不确定
- 没有方向性
- 尺度不变性差，固定选取三个像素的圆

ORB对FAST特征的改进

1. 对FAST角点计算Harris响应值，选取前N个具有最大响应值的作为最终角点集合。

1. 构造图像不同缩放尺度的图像金字塔

2. 在每层图像上产生FAST特征点，并都作为图像特征点

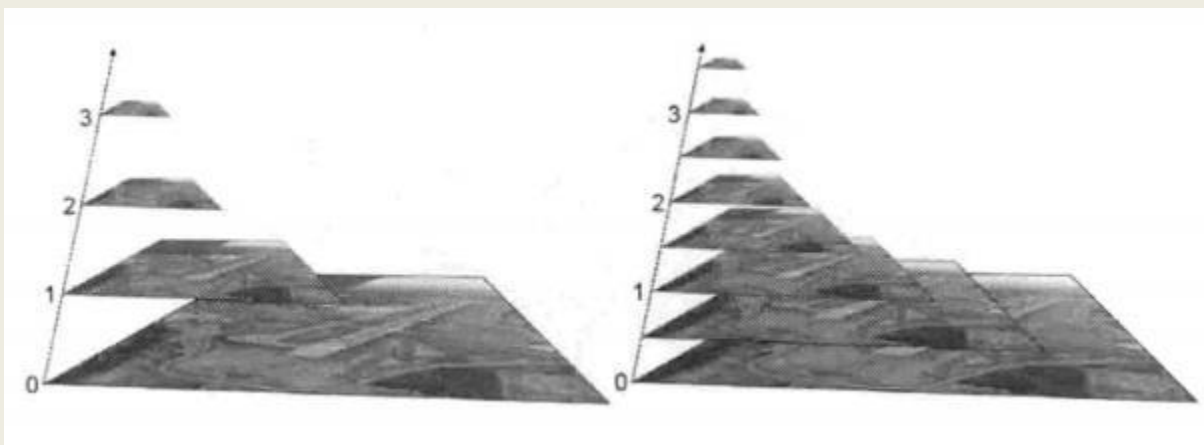
3. 计算Harris响应值，并选择N个响应值最大的作为ORB特征点

4. 基于矩计算ORB角点的方向，并旋转Brief描述子，产生具有旋转不变性的rotated brief描述子

ORB对FAST特征的改进：图像金字塔

1. 增加尺度不变性

构造图像金字塔，并在每一层上检测角点。



- 设置一个比例因子scaleFactor（默认为1.2）和金字塔的层数nlevels（默认为8）。
- 将原图像按比例因子缩小成nlevels幅图像。
- 缩放后的图像为： $I' = I / \text{scaleFactor}^k$ ($k=1, 2, \dots, \text{nlevels}$)。
- **nlevels幅不同比例的图像提取特征点总和作为这幅图像的FAST特征点**

https://sevenold.github.io/2019/03/cv-image_orb/

ORB对FAST特征的改进：FAST角点

1. 产生FAST角点之后。
2. 对FAST角点计算Harris响应值，选取前N个具有最大响应值的作为最终角点集合。

一小块局部点 (x,y) 沿着某个 u,v 方向变化的像素差异

$$E(u, v) = \sum_{x,y} w(x,y) [I(u+x, v+y) - I(x,y)]^2$$

对其进行近似

$$\begin{aligned} I(u+x, v+y) &\approx I(x,y) + uI_x + vI_y \\ E(u, v) &\approx \sum_{x,y} w(x,y) [uI_x + vI_y]^2 \\ &= \begin{bmatrix} u & v \end{bmatrix} \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} \end{aligned}$$

得到同 u,v 无关的二次型形式。 $\mathbf{x}^T \mathbf{A} \mathbf{x}$ ， $\mathbf{A}$ 的两个特征值都比较大时可以认为这一块是角点，当一个特征值比较大时是边，两个都很小时是平滑点。

如果 R 很大的正数认为是角点，很大的负数是边线，一般的正数是平滑区域。

ORB对FAST特征的改进: 前N响应值点

这一小块点变化
u,v个像素后的灰度
差异

$$E(u, v) \approx \sum_{x,y} w(x, y) [uI_x + vI_y]^2$$
$$= \begin{bmatrix} u & v \end{bmatrix} \sum_{x,y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

二次型矩阵A

A矩阵的两个特征值越大，则差异就越大。

Harris响应值不必直接求特征值，而是求：

$$|A| - k(\text{trace}(A)) = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$$

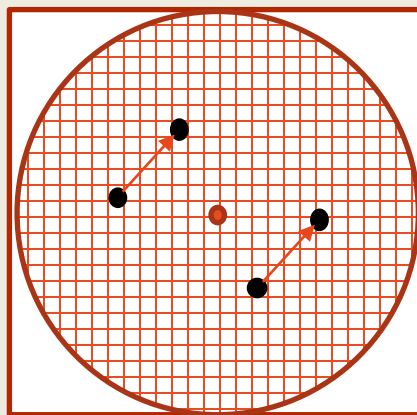
$$R = \det \left(\sum_{x,y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \right) - k \left(\text{trace} \left(\sum_{x,y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \right) \right)^2$$

$$\lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2 = (1 - 2k) \lambda_1 \lambda_2 - k\lambda_1^2 - k\lambda_2^2 \quad k \text{一般取} 0.04-0.06$$

当R越大，越可能是角点。ORB选择前N个Harris响应值最大的作为ORB特征点

ORB对FAST 的改进: BRIEF 描述子

- ORB采用BRIEF算法来计算一个特征点的描述子。
- BRIEF算法的**核心思想**是在关键点P的周围以一定模式选取N个点对，把这N个点对的比较结果组合起来作为描述子。
- 在一个特征点的邻域内，选择**n对 像素点** $\{p_i, q_i\}$, $i=1, \dots, n$
- 比较每个点对的灰度值的大小，如果 $p_i > q_i$ 则 $b_i=1$ ，否则 $b_i=0$
- 所有的点对都进行比较，则**生成长度为n的二进制串B**。一般n取128、256或512。



对于每个角点，点对的位置都是取相同的。固定模式的

ORB对FAST 的改进: BRIEF 描述子

ORB使用了一种基于学习的方法来选择一定数量的随机点对。

p_1

q_1

Brief描述子是256维的0-1向量，其后用海明距离进行描述子距离计算。

```
150 static int bit_pattern_31_[256*4] =
151 {
152     8,-3, 9,5/*mean (0), correlation (0)*/,
153     4,2, 7,-12/*mean (1.12461e-05), correlation (0.0437584)*/,
154     -11,9, -8,2/*mean (3.37382e-05), correlation (0.0617409)*/,
155     7,-12, 12,-13/*mean (5.62303e-05), correlation (0.0636977)*/,
156     2,-13, 2,12/*mean (0.000134953), correlation (0.085099)*/,
157     1,-7, 1,6/*mean (0.000528565), correlation (0.0857175)*/,
158     -2,-10, -2,-4/*mean (0.0188821), correlation (0.0985774)*/,
159     -13,-13, -11,-8/*mean (0.0363135), correlation (0.0899616)*/,
160     -13,-3, -12,-9/*mean (0.121806), correlation (0.099849)*/,
161     10,4, 11,9/*mean (0.122065), correlation (0.093285)*/,
162     -13,-8, -8,-9/*mean (0.162787), correlation (0.0942748)*/,
163     -11,7, -9,12/*mean (0.21561), correlation (0.0974438)*/,
164     7,7, 12,6/*mean (0.160583), correlation (0.130064)*/,
165     -4,-5, -3,0/*mean (0.228171), correlation (0.132998)*/,
166     -13,2, -12,-3/*mean (0.00997526), correlation (0.145926)*/,
167     -9,0, -7,5/*mean (0.198234), correlation (0.143636)*/,
168     12,-6, 12,-1/*mean (0.0676226), correlation (0.16689)*/,
169     -3,6, -2,12/*mean (0.166847), correlation (0.171682)*/,
170     -6,-13, -4,-8/*mean (0.101215), correlation (0.179716)*/,
171     11,-13, 12,-8/*mean (0.200641), correlation (0.192279)*/,
172     4,7, 5,1/*mean (0.205106), correlation (0.186848)*/,
173     5,-3, 10,-3/*mean (0.234908), correlation (0.192319)*/,
174     3,-7, 6,12/*mean (0.0709964), correlation (0.210872)*/,
175     -8,-7, -6,-2/*mean (0.0939834), correlation (0.212589)*/,
176     -2,11, -1,-10/*mean (0.127778), correlation (0.20866)*/,
177     -13,12, -8,10/*mean (0.14783), correlation (0.206356)*/,
178     -7,3, -5,-3/*mean (0.182141), correlation (0.198942)*/,
179     -4,2, -3,7/*mean (0.188237), correlation (0.21384)*/,
```

ORB对Brief的改进

- 增加特征点的局部方向信息，以产生具有旋转不变性的描述子
 - ORB算法提出使用矩（moment）法来确定FAST特征点的方向。

1. 在一个小的图像块 B 中，定义图像块的矩为：

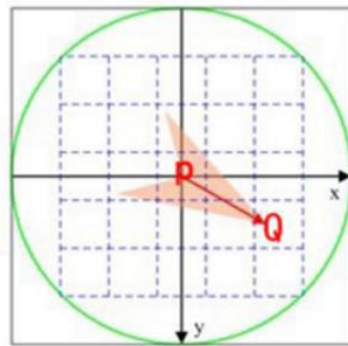
$$m_{pq} = \sum_{x,y \in B} x^p y^q I(x,y), \quad p, q = \{0, 1\}.$$

2. 通过矩可以找到图像块的质心：

$$C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right).$$

3. 连接图像块的几何中心 O 与质心 C ，得到一个方向向量 $\overrightarrow{OC}$ ，于是特征点的方向可以定义为：

$$\theta = \arctan(m_{01}/m_{10}).$$



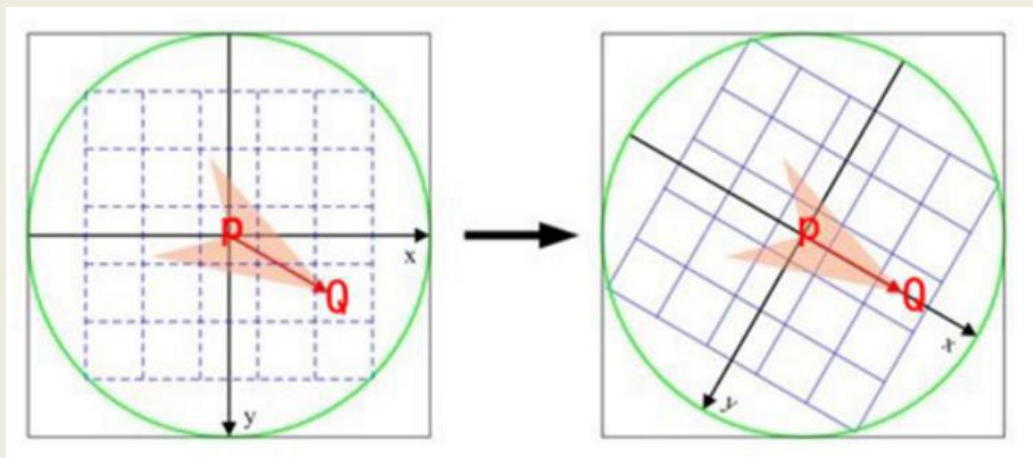
ORB对Brief的改进

如果原来取点对的位置为：

$$D = \begin{pmatrix} x_1, x_2, \dots, x_{2n} \\ y_1, y_2, \dots, y_{2n} \end{pmatrix}$$

根据计算的特征点的方向 θ ，根据特征点的方向旋转后取的点对位置为：

$$D_{\theta} = R(\theta) D$$



之后根据产生的点对产生描述子

ORB特征点和描述子总结

1. 改进了FAST

1. 固定了数量
2. 增加了尺度不变性

2. 改进Brief

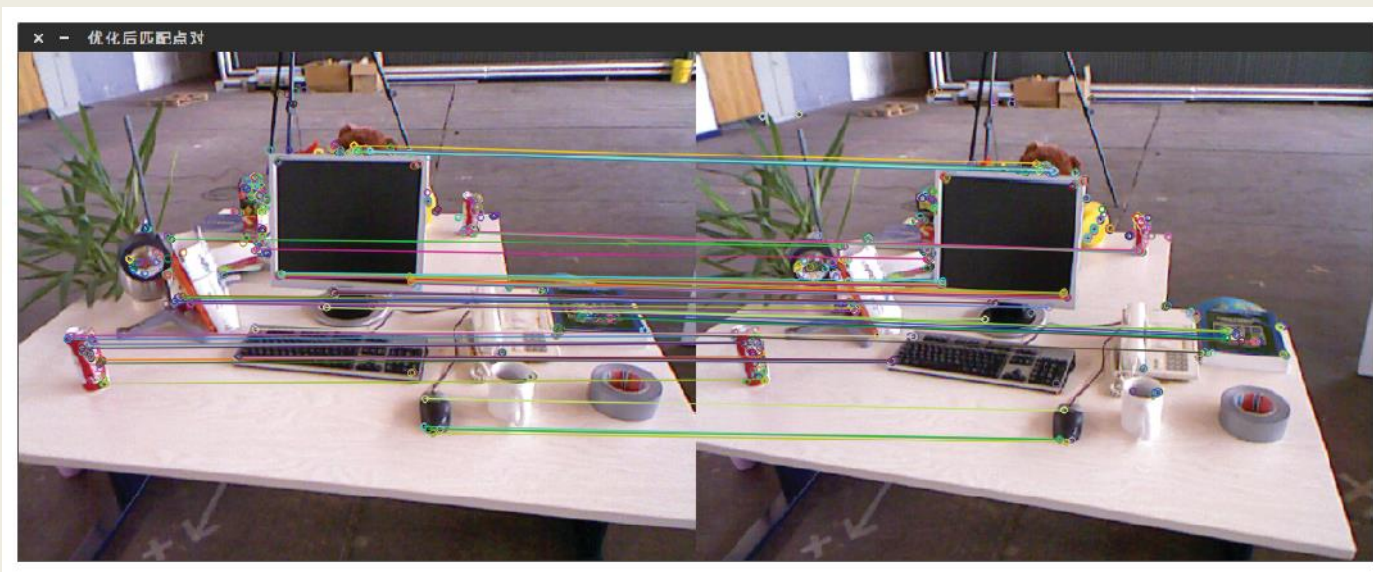
1. 考虑了特征点方向，通过旋转产生对方向不变的描述子

3. ORB的最终效果

1. 速度快、对尺度变化、方向变化增加了鲁棒性

特征匹配

- 特征匹配
 - 通过描述子的差异判断哪些特征为同一个点
 - 暴力匹配：比较图1中每个特征和图2特征的距离（逐一对比）
 - 加速：快速最近邻（FLANN）



特征点提取

//载入两幅图像

```
cv::Mat first_image = cv::imread(first_file, 0);  
cv::Mat second_image = cv::imread(second_file, 0);
```

//用FAST检测特征点，并且计算特征点的描述子

```
vector<cv::KeyPoint> keypoints1;  
cv::FAST(first_image, keypoints1, 40);  
vector<DescType> descriptor1;  
ComputeORB(first_image, keypoints1, descriptor1);
```

//暴力匹配，计算匹配点

```
vector<cv::DMatch> matches;  
BfMatch(descriptor1, descriptor2, matches);
```

//绘制匹配结果

```
cv::Mat image_show;  
cv::drawMatches(first_image, keypoints1, second_image, keypoints2, matches, image_show);  
cv::imshow("matches", image_show);  
cv::imwrite("matches.png", image_show);
```



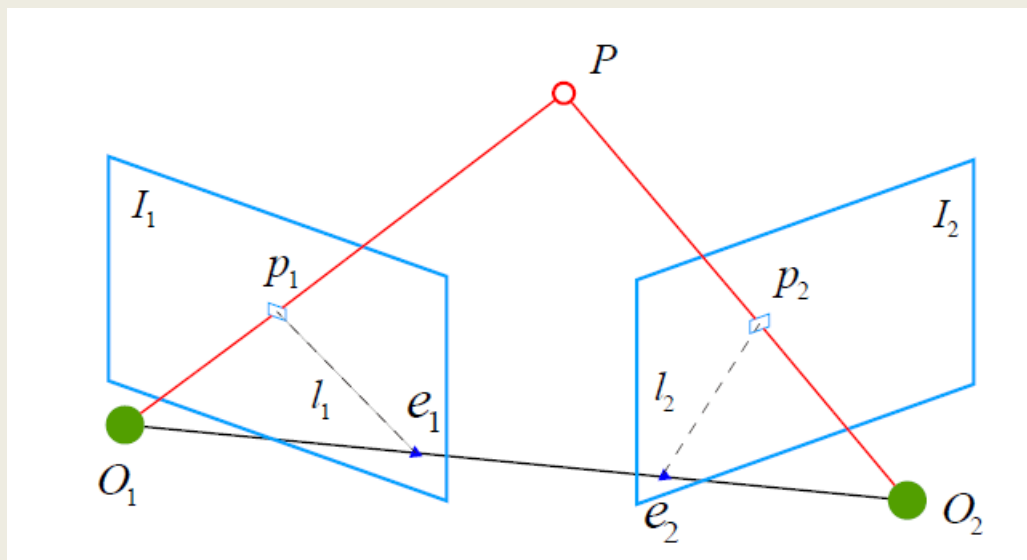
对极几何 (Epipolar Geometry)

王永才

2D-2D对极几何

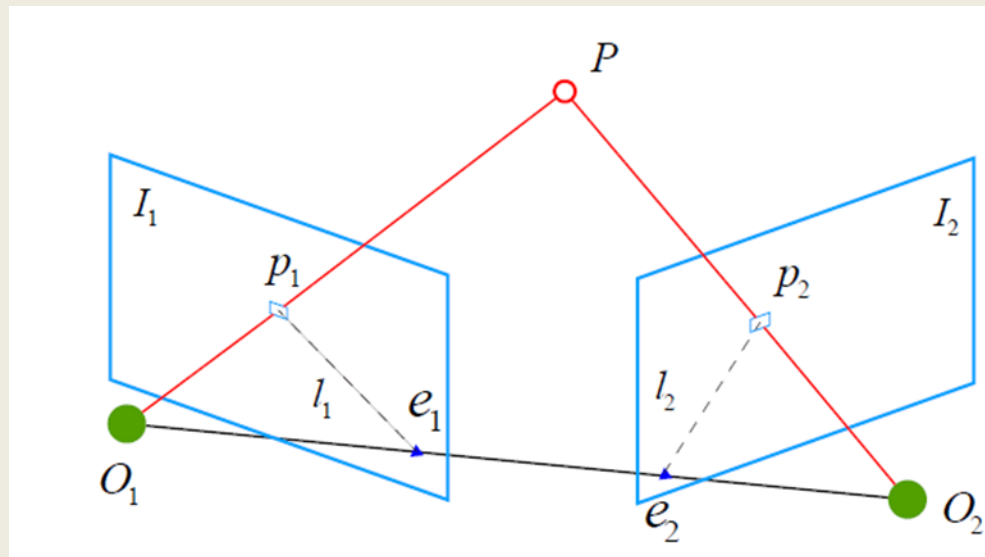
- 特征匹配之后，得到了特征点之间的对应关系
 - 如果只有两个单目图像，得到2D-2D间的关系 ——对极几何
 - 如果匹配的是地图和图像帧，得到3D-2D间的关系 ——PnP
 - 如果匹配的是RGB-D图，得到3D-3D间的关系 ——ICP

对极几何：得到相机在两个不同位置上拍摄的同一对象的两幅照片，计算相机的位姿变换



2D-2D对极几何

- 几何关系：
 - P 在两个图像的投影为 p_1, p_2
 - 两个相机之间的变换为 T_{12}
 - O_1P 在第二个图像上投影为 e_2p_2
 - 记 l_2 ，称为极线，反之亦然
 - e_1, e_2 称为极点
- 实践当中：
 - p_1, p_2 通过特征匹配得到， P 未知， e_1, e_2 未知
 - T_{12} 待求



2D-2D对极几何

- 世界坐标： $P = [X, Y, Z]^T$.
- 以第一个图为参考系，投影方程：

$$s_1 p_1 = K P, \quad s_2 p_2 = K (R P + t).$$

- 使用归一化坐标（去掉内参）：

$$x_1 = K^{-1} p_1, \quad x_2 = K^{-1} p_2.$$

- 齐次关系：

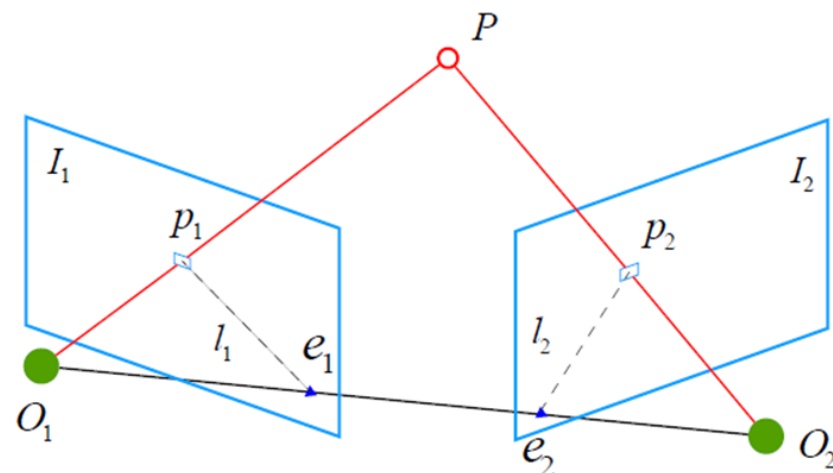
$$x_2 = R x_1 + t.$$

- 两侧左乘：

$$t^\wedge x_2 = t^\wedge R x_1.$$

- 再一步左乘：

$$x_2^T t^\wedge x_2 = x_2^T t^\wedge R x_1.$$



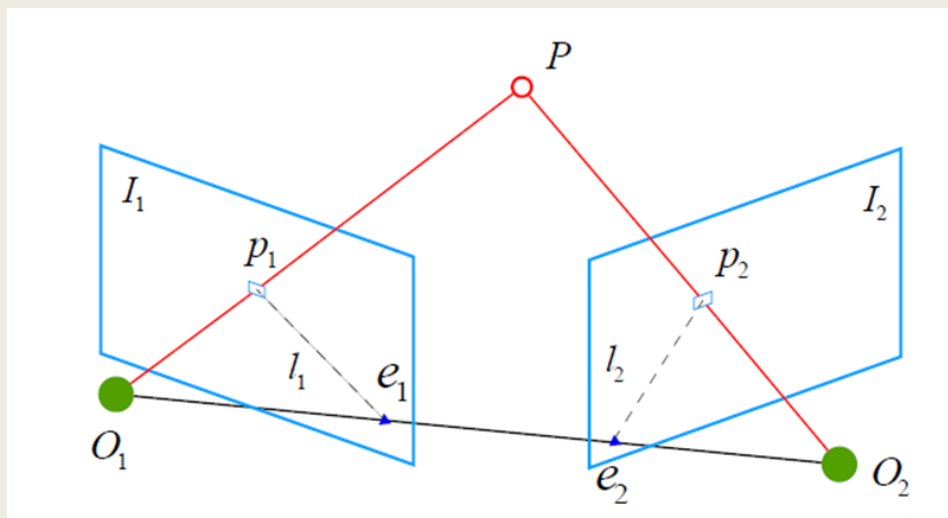
对极约束：

$$x_2^T t^\wedge R x_1 = 0.$$

带内参的形式：

$$p_2^T K^{-T} t^\wedge R K^{-1} p_1 = 0.$$

对极约束



对极约束：描述 O_1 , O_2 , P 在同一个平面上

$$x_2^T t^\wedge R x_1 = 0.$$

$$p_2^T K^{-T} t^\wedge R K^{-1} p_1 = 0.$$

定义：

$$E = t^\wedge R, \quad F = K^{-T} E K^{-1}$$

Essential矩阵 Fundamental矩阵

$$x_2^T E x_1 = p_2^T F p_1 = 0.$$

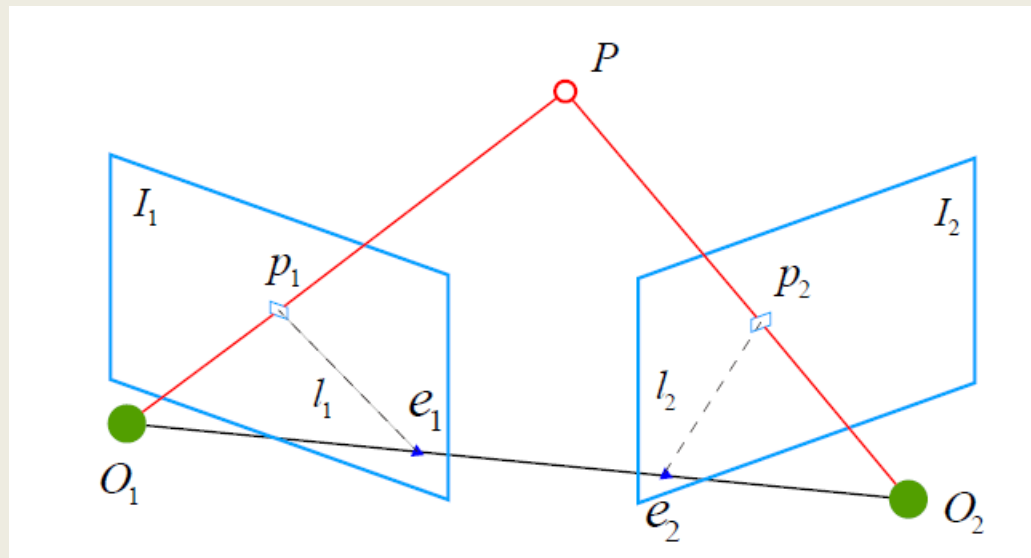
2D-2D对极几何

- 对极约束

$$E = t^{\wedge} R, \quad F = K^{-T} E K^{-1}$$

$$x_2^T E x_1 = p_2^T F p_1 = 0.$$

- 根据 p 的点对可以求 F
- 在已知 K 时可以求 E
- 再由 E 恢复 R, t
- 实际应用中 K 经常是已知的，所以我们考虑先求解 E ，然后恢复 t 和 R
 - E 的奇异值必然是 $[\sigma, \sigma, 0]^T$ 的形式，是本质矩阵的内在性质
 - E 具有尺度不变性，乘以任何非零常数对极约束仍然满足
 - t 和 R 各有3个自由度
 - 由于尺度等价性， E 实际具有5个自由度



2D-2D对极几何--八点法

- 八点法求E
 - 将E看成通常3x3的矩阵，去掉因子后剩八个自由度
 - 一对匹配点带来的约束：

$$\begin{pmatrix} u_1 & v_1 & 1 \end{pmatrix} \begin{pmatrix} e_1 & e_2 & e_3 \\ e_4 & e_5 & e_6 \\ e_7 & e_8 & e_9 \end{pmatrix} \begin{pmatrix} u_2 \\ v_2 \\ 1 \end{pmatrix} = 0.$$

我们使用8个点对来估计 E ，即八点法

- 向量形式：

$$e = [e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8, e_9]^T,$$

$$[u_1 u_2, u_1 v_2, u_1, v_1 u_2, v_1 v_2, u_2, v_2, 1] \cdot e = 0.$$

- 同理，对其他点对也可以列一个上述方程

2D-2D对极几何--八点法

八对点构成方程组

$$\begin{pmatrix} u_1^1 u_2^1 & u_1^1 v_2^1 & u_1^1 & v_1^1 u_2^1 & v_1^1 v_2^1 & v_1^1 & u_2^1 & v_2^1 & 1 \\ u_1^2 u_2^2 & u_1^2 v_2^2 & u_1^2 & v_1^2 u_2^2 & v_1^2 v_2^2 & v_1^2 & u_2^2 & v_2^2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ u_1^8 u_2^8 & u_1^8 v_2^8 & u_1^8 & v_1^8 u_2^8 & v_1^8 v_2^8 & v_1^8 & u_2^8 & v_2^8 & 1 \end{pmatrix} \begin{pmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \\ e_6 \\ e_7 \\ e_8 \\ e_9 \end{pmatrix} = 0.$$

求解线性方程组得到 E .

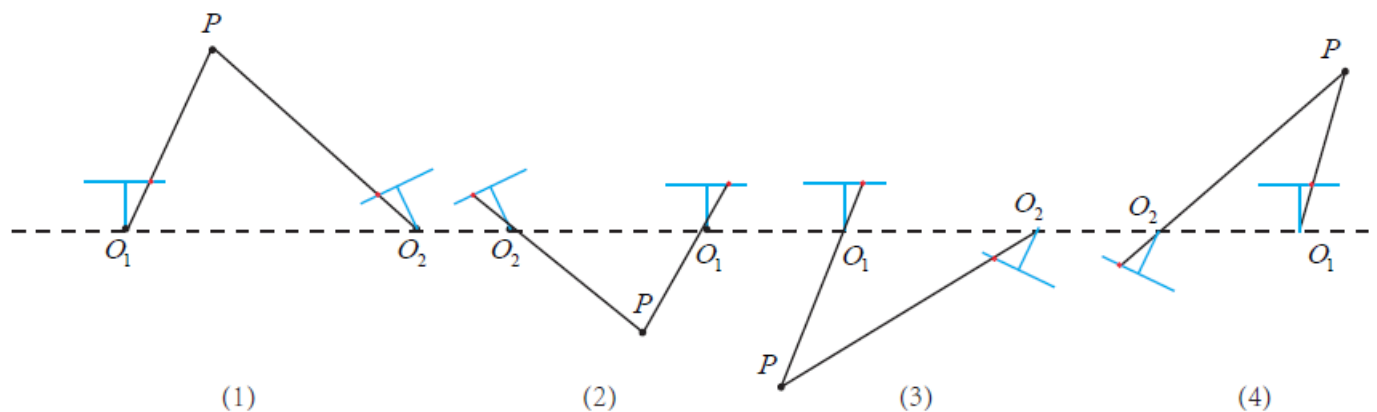
2D-2D对极几何--八点法

- 从E计算R, t : 奇异值分解

$$E = U\Sigma V^T,$$

$$\begin{aligned} t_1^\wedge &= UR_Z(\frac{\pi}{2})\Sigma U^T, & R_1 &= UR_Z^T(\frac{\pi}{2})V^T \\ t_2^\wedge &= UR_Z(-\frac{\pi}{2})\Sigma U^T, & R_2 &= UR_Z^T(-\frac{\pi}{2})V^T. \end{aligned}$$

四个可能的解，但只有一个深度为正



2D-2D对极几何--八点法

- SVD过程中：
 - 取 $E = U \text{diag}(\frac{\sigma_1 + \sigma_2}{2}, \frac{\sigma_1 + \sigma_2}{2}, 0) V^T$. 因为E的内在性质要求它的奇异值为 $\sigma, \sigma, 0$
- 最少可使用五个点计算R,t, 称为五点法
 - 但需要利用E的非线性性质，原理较复杂
- 多于八对点时：最小二乘或RANSAC

2D-2D对极几何——单应矩阵

- 从单应矩阵恢复 R, t :

- 八点法在特征点共面时会退化

- 设特征点位于某平面上 $n^T P + d = 0$. 或 $-\frac{n^T P}{d} = 1$.

- 两个图像特征点的坐标关系 :

$$\begin{aligned} p_2 &= K(RP + t) \\ &= K \left(RP + t \cdot \left(-\frac{n^T P}{d} \right) \right) \\ &= K \left(R - \frac{tn^T}{d} \right) P \\ &= K \left(R - \frac{tn^T}{d} \right) K^{-1} p_1. \end{aligned}$$

中间记为H



单应矩阵

$$p_2 = H p_1.$$

$$\begin{pmatrix} u_2 \\ v_2 \\ 1 \end{pmatrix} = \begin{pmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{pmatrix} \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix}.$$

2D-2D对极几何——单应矩阵

$$\begin{pmatrix} u_2 \\ v_2 \\ 1 \end{pmatrix} = \begin{pmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{pmatrix} \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix}.$$

去掉第三行：

$$u_2 = \frac{h_1 u_1 + h_2 v_1 + h_3}{h_7 u_1 + h_8 v_1 + h_9}$$

$$v_2 = \frac{h_4 u_1 + h_5 v_1 + h_6}{h_7 u_1 + h_8 v_1 + h_9}.$$

- 一对点提供两个约束
- 写成关于H的线性方程：
- 四对点构成8个方程
- 类似八点法
 - 先计算H
 - 再用H恢复R,t,n,d,K

$$\begin{pmatrix} u_1^1 & v_1^1 & 1 & 0 & 0 & 0 & -u_1^1 u_2^1 & -v_1^1 u_2^1 \\ 0 & 0 & 0 & u_1^1 & v_1^1 & 1 & -u_1^1 v_2^1 & -v_1^1 v_2^1 \\ u_1^2 & v_1^2 & 1 & 0 & 0 & 0 & -u_1^2 u_2^2 & -v_1^2 u_2^2 \\ 0 & 0 & 0 & u_1^2 & v_1^2 & 1 & -u_1^2 v_2^2 & -v_1^2 v_2^2 \\ u_1^3 & v_1^3 & 1 & 0 & 0 & 0 & -u_1^3 u_2^3 & -v_1^3 u_2^3 \\ 0 & 0 & 0 & u_1^3 & v_1^3 & 1 & -u_1^3 v_2^3 & -v_1^3 v_2^3 \\ u_1^4 & v_1^4 & 1 & 0 & 0 & 0 & -u_1^4 u_2^4 & -v_1^4 u_2^4 \\ 0 & 0 & 0 & u_1^4 & v_1^4 & 1 & -u_1^4 v_2^4 & -v_1^4 v_2^4 \end{pmatrix} \begin{pmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \end{pmatrix} = \begin{pmatrix} u_2^1 \\ v_2^1 \\ u_2^2 \\ v_2^2 \\ u_2^3 \\ v_2^3 \\ u_2^4 \\ v_2^4 \end{pmatrix}.$$

2D-2D对极几何——实践

```
Mat img_1 = imread ( argv[1], CV_LOAD_IMAGE_COLOR );
Mat img_2 = imread ( argv[2], CV_LOAD_IMAGE_COLOR );

vector<KeyPoint> keypoints_1, keypoints_2;
vector<DMatch> matches;
find_feature_matches ( img_1, img_2, keypoints_1, keypoints_2, matches );
```



```
Mat descriptors_1, descriptors_2;
// used in OpenCV3
Ptr<FeatureDetector> detector = ORB::create();
Ptr<DescriptorExtractor> descriptor = ORB::create();
Ptr<DescriptorMatcher> matcher = DescriptorMatcher::create ( "BruteForce-Hamming" );
//-- 第一步:检测 Oriented FAST 角点位置
detector->detect ( img_1, keypoints_1 );
detector->detect ( img_2, keypoints_2 );

//-- 第二步:根据角点位置计算 BRIEF 描述子
descriptor->compute ( img_1, keypoints_1, descriptors_1 );
descriptor->compute ( img_2, keypoints_2, descriptors_2 );

//-- 第三步:对两幅图像中的BRIEF描述子进行匹配, 使用 Hamming 距离
vector<DMatch> match;
//BFMatcher matcher ( NORM_HAMMING );
matcher->match ( descriptors_1, descriptors_2, match );
```

2D-2D对极几何——实践

```
//-- 估计两张图像间运动
Mat R,t;
pose_estimation_2d2d ( keypoints_1, keypoints_2, matches, R, t );

for ( int i = 0; i < ( int ) matches.size(); i++ )
{
    points1.push_back ( keypoints_1[matches[i].queryIdx].pt );
    points2.push_back ( keypoints_2[matches[i].trainIdx].pt );
}

//-- 计算基础矩阵
Mat fundamental_matrix;
fundamental_matrix = findFundamentalMat ( points1, points2, CV_FM_8POINT );
cout<<"fundamental_matrix is "<<endl<< fundamental_matrix<<endl;

//-- 计算本质矩阵
Point2d principal_point ( 325.1, 249.7 ); //相机光心, TUM dataset标定值
double focal_length = 521; //相机焦距, TUM dataset标定值
Mat essential_matrix;
essential_matrix = findEssentialMat ( points1, points2, focal_length, principal_point );
cout<<"essential_matrix is "<<endl<< essential_matrix<<endl;
```

结果

fundamental_matrix is

```
[4.544437503975349e-06, 0.0001333855576996833, -0.01798499246473872;  
-0.0001275657012968023, 2.266794804651179e-05, -0.01416678429218289;  
0.01814994639967876, 0.004146055871077865, 1]
```

essential_matrix is

```
[-0.01605869979668707, -0.3478480375843301, -0.1087969472780801;  
0.3402378250881326, -0.03711533924994918, 0.6110343523877446;  
0.06664856250675474, -0.6096197642612388, -0.0180690753538746]
```

homography_matrix is

```
[0.9421806766565817, -0.1641507156395019, 36.55445058662729;  
0.04136405926722522, 0.9527219670853346, 7.936203381087487;  
-2.526780556700765e-05, 9.250957414964784e-06, 1]
```

R is

```
[0.9985579165439813, -0.0511742787639671, 0.0162259206312298;  
0.05062830761925086, 0.9981905435835956, 0.03244091811118115;  
-0.01785670112181209, -0.03157264469875086, 0.9993419366421946]
```

t is

```
[-0.8566295409014812;  
-0.1378237582838886;  
0.4971825030181793]
```

$t^R =$

```
[-0.02271043104655323, -0.4919314123966253, -0.1538621183854518;  
0.481168946671962, -0.05248901613935576, 0.8641330682226095;  
0.09425530100972325, -0.8621325385089328, -0.02555353142499148]
```

epipolar constraint = [-0.001450557017811854]

小结

- 2D-2D情况下，只知道图像坐标之间的对应关系
 - 当特征点在平面上时（例如俯视或仰视），使用 H 恢复 R, t
 - 否则，使用 E 或 F 恢复 R, t
- 求得 R, t 后：
 - 利用三角化计算特征点的3D位置（即深度）